Ifosup Wavre

5IDEV — Projet d'intégration de développement

Projet d'intégration 1 — Rapport de projet

Canopée

Messagerie en temps réel (B2B)

Coisne Valentin · van der Veen Georgé

Sous la supervision de
Thibault Six

Année académique 2026-2027

Membres du groupe

Nom & prénom	Rôle(s) dans l'équipe	Contribution principale
Coisne Valentin	Frontend, VPS & déploiement	Pages React (accueil, tarifs, équipe, connexion, inscription, chat, profil), CI front, Nginx/systemd, déploiement continu, refresh du token, toast, suppression de compte, liste des membres d'un groupe
van der Veen Georgé	Backend, temps réel, documentation	Modèles et API Django REST, WebSockets (Django Channels), groupes et renommage, tests et CI backend, devlog, rapports

URL de production https://canopee.coisnevalentin.be
Repo GitHub github.com/AlphaXZero/Integration-Project-1-Real-Time-Messaging

Table des matières

1 Introduction 4
1.1 Choix de la stack technique 4
2 Cahier des charges 6
2.1 Acteurs du système 6
2.2 MVP — exigences fonctionnelles 6
2.3 Exigences non fonctionnelles 7
2.4 Fonctionnalités optionnelles retenues 7
3 Architecture & modèle de données 8
3.1 Schéma d'architecture 8
3.2 Diagramme de classes UML 8
3.3 Tables & relations 9
3.4 Intégrité & sécurité des données 9
3.5 Fonctionnement du temps réel 10
4 Fonctionnalités implémentées 11
4.1 MVP obligatoire 11
4.2 Fonctionnalités optionnelles 15
4.3 Landing page & pricing 18
5 CI/CD & déploiement 19
5.1 Workflow Git : ce que nous avons appris 19
5.2 Intégration continue (à chaque Pull Request) 20
5.3 Déploiement continu (au merge sur `main`) 21
6 Qualité, tests & gestion des erreurs 22
7 Difficultés rencontrées & solutions 23
8 Utilisation de l'IA 25
9 Réflexion, améliorations & conclusion 25
9.1 Glossaire 26

Table des figures

Fig. 1 Architecture de Canopée et chemin d'un message d'Alice à Bob. 8
Fig. 2 Diagramme de classes du modèle de données. 8
Fig. 3 Conversation privée : mes messages à droite en vert, ceux de l'autre à gauche avec son pseudo. 11
Fig. 4 Conversation de groupe nommée « Projet Canopée » (4 membres) : (1) renommer le groupe, (2) ajouter un participant. 12
Fig. 5 Création d'une conversation : un pseudo donne une conversation privée ; dès qu'il y en a plusieurs, le champ « Nom du groupe » apparaît. 12
Fig. 6 Deux navigateurs côte à côte : Valentin (à gauche) envoie un message, il apparaît immédiatement chez Georgé (à droite), sans rechargement. 13
Fig. 7 Georgé est dans « Projet Canopée » quand Marie lui écrit en privé : (1) toast (avec un son), (2) point « non lu » sur la conversation. 13
Fig. 8 Mauvais identifiants : message clair. 14
Fig. 9 Inscription avec un pseudo déjà pris. 14
Fig. 10 Accueil sur mobile. 14
Fig. 11 Liste des conversations. 14
Fig. 12 Conversation ouverte. 14
Fig. 13 Renommage d'un groupe (✎). Le nouveau nom apparaît en direct chez tous les membres ; un nom vide réaffiche la liste des membres. 15
Fig. 14 Le bouton « membres » affiche les participants du groupe ; un compte supprimé apparaît barré. 15
Fig. 15 Dans la conversation privée avec Sophie, le « + » propose de créer un groupe : ajout de Lucas… 16
Fig. 16 …ce qui crée et ouvre un nouveau groupe « sophie, lucas » ; la conversation privée avec Sophie ne change pas. 16
Fig. 17 Page profil avec « Supprimer mon compte » (le mot de passe est demandé pour confirmer). 17
Fig. 18 Thomas a supprimé son compte : la zone d'envoi est remplacée par « Cette personne a quitté Canopée ». 17
Fig. 19 Landing page : promesse du produit et appels à l'action « Créer un compte » / « Se connecter ». 18
Fig. 20 Grille tarifaire avec bascule mensuel / annuel. 18
Fig. 21 Commits par personne et par semaine (hors commits de merge). 19
Fig. 22 Pipeline CI/CD : de la branche de fonctionnalité à la production. 21

1 Introduction

Canopée est une application web de messagerie en temps réel pour les équipes, dans l'esprit de Slack, Teams ou Discord, en version professionnelle (B2B). Le nom vient de l'idée de « réunir toute l'équipe sous la même canopée ». Le projet est réalisé à deux dans le cadre du cours de projet d'intégration, mais il a été pensé comme un vrai produit : il est déployé en production, avec une landing page et une grille tarifaire.

Les objectifs étaient de permettre à des utilisateurs de créer un compte, de discuter en privé (1:1) ou en groupe, de recevoir les messages **en temps réel**, sans recharger la page, d'être notifiés des nouveaux messages, de conserver tout l'historique, et de contrôler strictement qui a accès à quelle conversation.

Le second objectif, tout aussi important pour nous, était d'apprendre à travailler comme une vraie équipe de développement : un dépôt Git partagé, une branche par fonctionnalité, des Pull Requests relues, une intégration continue qui teste chaque modification, et un déploiement automatique sur notre serveur.

Périmètre. Le tableau suivant résume ce qui est fait et ce qui ne l'est pas.

Fonctionnalité	État
Inscription, connexion, déconnexion (JWT), suppression de compte	**Fait**
Conversations privées (1:1) et de groupe, sans doublon de conversation privée	**Fait**
Ajout d'un participant, nom de groupe, renommage et liste des membres	**Fait**
Messages, nouvelles conversations et changements reçus en temps réel	**Fait**
Notifications : toast avec son et indicateur « non lu » par conversation	**Fait**
Compteur **chiffré** de non-lus, conservé après rechargement	**Partiel**
Interface responsive, landing page, tarifs, page équipe, profil	**Fait**
CI (tests, lint, build) et déploiement continu sur un VPS en HTTPS	**Fait**
Rôle d'administrateur de groupe, quitter un groupe	**Non fait**
Fonctionnalités optionnelles lourdes (fichiers, recherche, co-édition, chatbot)	**Non fait**

1.1 Choix de la stack technique

Composant	Technologie
Frontend	React 19 (Vite), React Router, Tailwind CSS
Backend	Python, Django 6 + Django REST Framework
Authentification	JWT (`djangorestframework-simplejwt`) : token d'accès de 30 min + refresh de 3 jours
Temps réel	Django Channels (WebSockets) servi par Daphne (ASGI)
Base de données	SQLite (fichier sur le disque du serveur, sauvegardé à chaque déploiement)
Hébergement	VPS Debian, Nginx (HTTPS Let's Encrypt), service systemd
CI/CD	GitHub Actions

Pourquoi Django et Django REST Framework ? Le cours impose un langage interprété pour le backend. Django apporte d'origine tout ce dont une messagerie a besoin : un système d'utilisateurs avec mots de passe hachés, un ORM qui protège contre les injections SQL, des migrations pour faire évoluer la base, et une interface d'administration qui nous a servi à créer des données de test avant d'avoir l'API. Django REST Framework ajoute les **serializers** (validation des entrées, conversion en JSON) et les permissions par requête.

Pourquoi Django Channels pour le temps réel ? Le temps réel est au cœur du sujet. Channels ajoute les WebSockets **dans le même projet Django** : le code temps réel réutilise nos modèles, notre authentification JWT et nos règles d'accès, sans service séparé à maintenir (par exemple un serveur Node.js ou un service tiers payant comme Pusher). Les alternatives ont été écartées : le **polling** (redemander les messages toutes les secondes) gaspille des requêtes et ajoute du délai, et les Server-Sent Events ne vont que du serveur vers le client.

Pourquoi React séparé du backend ? Le frontend et le backend sont deux applications indépendantes qui communiquent en JSON. Ce découpage correspondait à notre répartition des rôles (Valentin au frontend, Georgé au backend) et permet de travailler en parallèle sans se gêner.

Pourquoi JWT plutôt que des cookies de session ? Nous avons d'abord essayé les cookies de session. Avec un frontend et un backend sur des origines différentes en développement (`localhost:5173` et `localhost:8000`), ils demandaient de configurer les cookies cross-origin, SameSite et CSRF. Le JWT, envoyé dans un en-tête `Authorization`, évite tout cela.

Pourquoi un VPS ? L'analyse d'hébergement du premier cours nous a orientés vers un VPS plutôt qu'un hébergeur « platform as a service » gratuit, pour trois raisons : un **disque persistant** (indispensable pour une base SQLite), des **connexions WebSocket longues** sans coupure ni mise en veille, et un **contrôle total** sur Nginx, le service et les déploiements. C'était aussi l'occasion d'apprendre à administrer un serveur.

Pourquoi SQLite en production ? Pour le nombre d'utilisateurs visé par ce projet (quelques dizaines), SQLite est largement assez solide : c'est une base transactionnelle complète, sans serveur à administrer, et la sauvegarde se résume à copier un fichier (ce que fait notre script de déploiement avant chaque mise à jour). Le backend ne tourne qu'en un seul processus Daphne, donc il n'y a pas d'écritures concurrentes venant de plusieurs serveurs. Si l'application devait grandir, Django permet de passer à PostgreSQL en changeant une seule configuration.

Et la couche de canaux en mémoire ? Pour faire passer un message d'une connexion à une autre, Channels utilise une « channel layer ». Nous utilisons la version en mémoire, qui fonctionne parfaitement tant qu'il n'y a qu'un seul processus serveur, ce qui est notre cas. Sa seule limite : avec plusieurs processus ou plusieurs serveurs, il faudrait la remplacer par Redis.

2 Cahier des charges

2.1 Acteurs du système

Acteur	Peut	Ne peut pas
Visiteur (non connecté)	Voir la landing page, les tarifs et l'équipe, créer un compte, se connecter	Accéder au chat, lire ou envoyer un message (redirigé vers la connexion)
Utilisateur connecté	Voir ses conversations, en créer (privée ou groupe), voir son profil, se déconnecter, supprimer son compte	Voir les conversations dont il ne fait pas partie
Participant d'une conversation	Lire l'historique, envoyer des messages, ajouter quelqu'un, renommer le groupe	Écrire à une personne dont le compte est supprimé
Administrateur (staff Django)	Gérer les données via `/admin/`	—

Les acteurs interagissent uniquement à travers les conversations : un utilisateur en crée une avec d'autres utilisateurs (par leur pseudo), et tous les participants deviennent égaux. Il n'y a pas de rôle d'« administrateur de groupe » : tout participant peut ajouter quelqu'un ou renommer le groupe. C'est un choix de simplicité, adapté à de petites équipes qui se font confiance.

Toutes les permissions sont vérifiées **par le backend, à chaque requête** : le frontend n'est jamais considéré comme fiable.

2.2 MVP — exigences fonctionnelles

User story	État
En tant que visiteur, je crée un compte avec un pseudo unique et un mot de passe.	**Fait**
En tant qu'utilisateur, je me connecte, je me déconnecte, et je reste connecté sans retaper mon mot de passe pendant 3 jours.	**Fait**
En tant qu'utilisateur, je vois la liste de mes conversations, avec leur nom et leur type (privée ou groupe).	**Fait**
En tant qu'utilisateur, je démarre une conversation privée avec un collègue ; si elle existe déjà, elle s'ouvre au lieu d'être dupliquée.	**Fait**
En tant qu'utilisateur, je crée une conversation de groupe avec plusieurs collègues et je lui donne un nom.	**Fait**
En tant que participant, j'ouvre une conversation et je lis tout son historique, dans l'ordre.	**Fait**
En tant que participant, j'envoie un message, et les autres le reçoivent sans recharger la page.	**Fait**
En tant que participant, je suis notifié (toast + son + indicateur) quand un message arrive dans une autre conversation.	**Fait**
En tant que participant, je vois combien de messages je n'ai pas lus dans chaque conversation, même après avoir rechargé la page.	**Partiel**
En tant qu'utilisateur, je ne peux jamais lire ni écrire dans une conversation dont je ne fais pas partie.	**Fait**

L'indicateur de non-lus existe (un point doré à côté de la conversation), mais ce n'est pas encore un **compteur** et il est perdu au rechargement de la page. La section 9 explique comment nous le ferions.

2.3 Exigences non fonctionnelles

- **Sécurité.** Mots de passe hachés par Django, accès à l'API par JWT, HTTPS en production, permissions vérifiées à chaque requête, secrets dans des variables d'environnement, `DEBUG` désactivé en production (aucune trace d'erreur n'est montrée à l'utilisateur).
- **Temps réel.** Un message doit apparaître chez les autres participants en moins d'une seconde, sans **polling** : le serveur le pousse par WebSocket.
- **Fiabilité du temps réel.** Si la connexion WebSocket se coupe (redémarrage du serveur, réseau instable, ordinateur en veille), le frontend se reconnecte tout seul, avec un délai croissant (1 s, 2 s, 4 s… jusqu'à 30 s), puis recharge ce qui a été manqué.
- **Persistance.** Les messages survivent à une déconnexion, à un redémarrage du backend et à un redéploiement : ils sont dans la base SQLite, sur le disque du serveur, et la base est sauvegardée avant chaque déploiement.
- **Disponibilité.** Le backend tourne comme un service systemd qui redémarre automatiquement en cas de crash. Chaque déploiement vérifie que le site répond avant de se terminer.
- **Responsive.** L'interface est utilisable sur ordinateur et sur téléphone (voir la section 4.1).
- **Messages d'erreur clairs.** Chaque formulaire affiche une erreur compréhensible, jamais une erreur technique.

2.4 Fonctionnalités optionnelles retenues

Nous avons préféré soigner le MVP plutôt que commencer une grosse fonctionnalité optionnelle (co-édition, fichiers, chatbot…) sans pouvoir la finir. Les extensions que nous avons ajoutées ont toutes une valeur directe pour une équipe en entreprise :

- **Gestion des groupes.** Nommer et renommer un groupe, voir ses membres, ajouter un collègue. Ajouter quelqu'un à une conversation **privée** ne la modifie pas : cela crée un **nouveau groupe**, pour que personne ne lise un échange privé auquel il n'a pas participé (même comportement que Messenger ou Teams).
- **Suppression de compte.** Indispensable en entreprise (départ d'un employé) et pour le RGPD. Le compte est désactivé, mais ses messages restent lisibles par les autres, pour ne pas casser l'historique des conversations.
- **Notifications enrichies.** Un toast avec un son quand un message arrive dans une autre conversation, pour ne rien manquer en travaillant.

3 Architecture & modèle de données

3.1 Schéma d'architecture

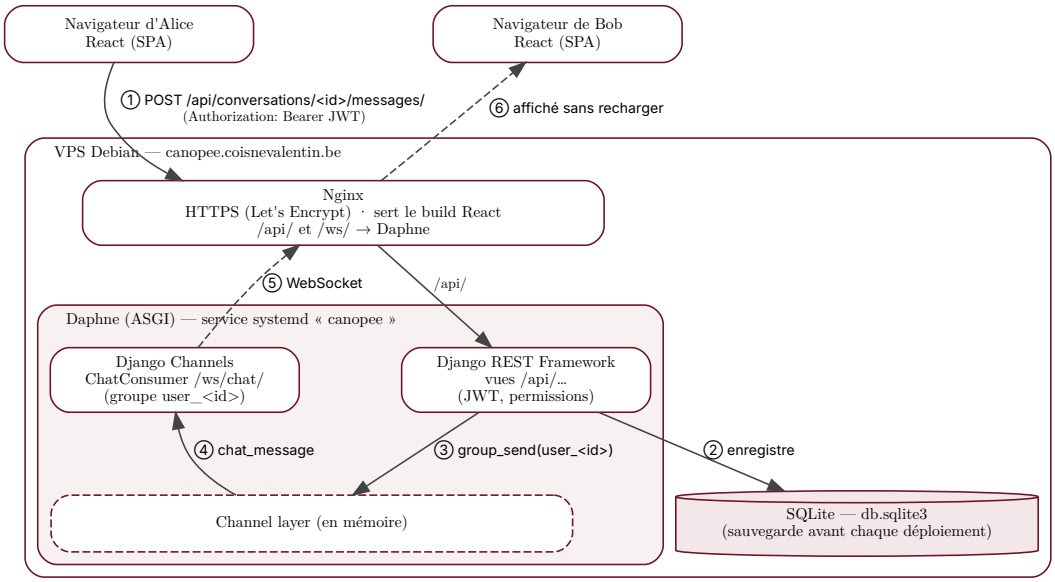


Fig. 1. – Architecture de Canopée et chemin d'un message d'Alice à Bob.

Le frontend React est compilé en fichiers statiques et servi par **Nginx**, qui gère aussi le HTTPS. Nginx transmet les requêtes `/api/` et les connexions `/ws/` à **Daphne**, le serveur ASGI qui fait tourner Django. Il n'y a aucune intégration externe : tout tourne sur notre VPS.

Chemin d'un message (numéros du schéma) :

1. Alice envoie son message avec une requête HTTP `POST /api/conversations/<id>/messages/`, avec son token JWT.
2. La vue Django vérifie qu'Alice fait partie de la conversation, puis enregistre le message dans SQLite.
3. La vue envoie le message au groupe personnel de **chaque** participant (`user_<id>`) via la channel layer.
4. Le consumer WebSocket de chaque participant connecté reçoit l'événement…
5. … et le transmet au navigateur par la connexion WebSocket déjà ouverte.
6. Le message apparaît chez Bob sans qu'il ait rien demandé. Si Bob est dans une autre conversation, il reçoit un toast.

3.2 Diagramme de classes UML

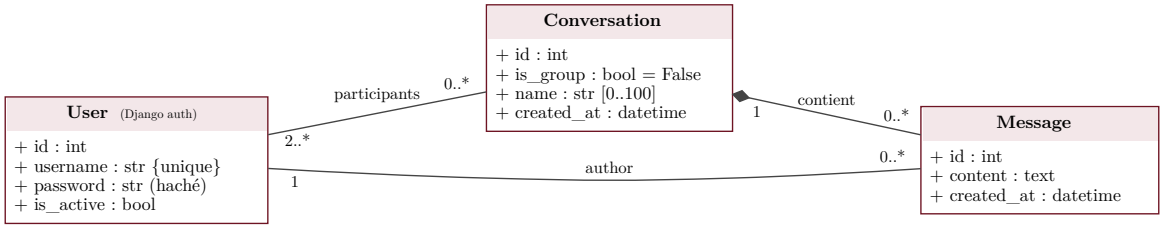


Fig. 2. – Diagramme de classes du modèle de données.

`User` est le modèle d'utilisateur fourni par Django : nous n'avons pas eu besoin de l'étendre. Une conversation a au moins deux participants, et un utilisateur peut participer à zéro ou plusieurs conversations (relation N-N). Un message a exactement un auteur et appartient à exactement une conversation. Le losange (composition) indique qu'un message n'existe pas sans sa conversation : supprimer une conversation supprime ses messages.

3.3 Tables & relations

- **`auth_user`** (Django) : les comptes. `username` est unique, `password` est un hash (jamais le mot de passe en clair), `is_active` passe à faux quand le compte est supprimé.
- **`chat_conversation`** : une conversation. `is_group` distingue les conversations privées des groupes, `name` est le nom optionnel d'un groupe, `created_at` est rempli automatiquement.
- **`chat_conversation_participants`** : la table de liaison de la relation **N-N** entre utilisateurs et conversations. Django la crée automatiquement à partir du `ManyToManyField`. Ce choix permet de gérer les conversations privées et les groupes avec la même structure.
- **`chat_message`** : un message, avec son contenu, sa date et deux clés étrangères (relations **1-N**) : son auteur et sa conversation.

Migrations. Le schéma évolue uniquement par des migrations Django, versionnées dans Git et appliquées automatiquement à chaque déploiement :

- `0001_initial` crée les conversations et les messages ;
- `0002` ajoute `is_group` (faux par défaut) et `name` ;
- `0003` est une **migration de données** : elle marque comme groupes les conversations créées avant `0002` qui ont plus de deux participants (voir la difficulté n°6).

Données de test. Nous n'avons pas écrit de **seeders** : les données de test sont créées dans l'admin Django ou dans le shell Django (`manage.py shell`). Les tests automatiques créent leurs propres données dans une base temporaire.

3.4 Intégrité & sécurité des données

Les règles de la base, en français :

- un message appartient toujours à une conversation et a toujours un auteur ;
- supprimer une conversation supprime ses messages (`on_delete=CASCADE`) ;
- une conversation privée a exactement deux participants, et il n'en existe qu'une seule par paire de personnes ;
- seuls les groupes peuvent avoir un nom, d'au plus 100 caractères ;
- deux comptes ne peuvent pas avoir le même pseudo (contrainte d'unicité) ;
- un compte supprimé n'est pas effacé mais **désactivé** (une forme de « soft delete ») : il ne peut plus se connecter, on ne peut plus lui écrire ni l'ajouter à une conversation, mais ses anciens messages restent visibles. Sans cela, la suppression en cascade effacerait aussi ses messages dans les conversations des autres.

Contrôle d'accès. Chaque vue ne récupère une conversation **que si l'utilisateur connecté en fait partie** :

```
conversation = get_object_or_404(
    Conversation, id=conversation_id, participants=request.user
)
```

Pour une personne extérieure, une conversation qui n'existe pas et une conversation qui n'est pas la sienne donnent la même réponse : une erreur 404. Une erreur 403 confirmerait que la conversation existe, ce qui est déjà une fuite d'information.

Validation des entrées. Les **serializers** de Django REST Framework valident tout ce qui arrive (champs obligatoires, pseudos existants, longueur du nom…). Les champs sensibles sont en **lecture seule** : le client ne peut envoyer que le contenu d'un message, jamais son auteur ni sa conversation, qui sont fixés par le serveur. Impossible donc de poster au nom de quelqu'un d'autre. De même, c'est le serveur qui décide si une conversation est un groupe, en comptant les participants.

Injections et XSS. L'ORM de Django génère des requêtes SQL paramétrées : aucune entrée utilisateur n'est collée dans une requête, ce qui empêche les injections SQL. Côté frontend, React échappe automatiquement le texte affiché : un message contenant du HTML s'affiche comme du texte et n'est jamais exécuté.

Compromis connu. Les tokens JWT sont stockés dans le `localStorage` du navigateur, lisible par JavaScript. Grâce à l'échappement de React, le risque de vol par une faille XSS est faible ; des cookies `HttpOnly` seraient plus sûrs, mais demanderaient de gérer le CSRF.

3.5 Fonctionnement du temps réel

Une connexion par utilisateur. À l'ouverture de la page de chat, le frontend ouvre **une seule** connexion WebSocket (`wss://…/ws/chat/`), et non une par conversation. Côté serveur, le consumer range cette connexion dans le groupe personnel de l'utilisateur, `user_<id>`. Un utilisateur reçoit ainsi les événements de **toutes** ses conversations, même celles qui ne sont pas ouvertes : c'est ce qui permet les notifications.

Authentification. Le navigateur ne peut pas ajouter d'en-tête à une connexion WebSocket, donc le token JWT est passé dans l'URL (`?token=…`). Le consumer le vérifie et refuse la connexion s'il est absent, invalide ou expiré.

Envoi par HTTP, réception par WebSocket. L'envoi d'un message reste une requête `POST` classique, déjà validée, sécurisée et testée. Le WebSocket ne sert qu'à **recevoir**. Après chaque action, la vue envoie un événement au groupe de chaque participant concerné :

Action	Événement envoyé au navigateur	Effet dans le frontend
Nouveau message	`{"type": "message", "data": {…}}`	Ajouté à la conversation ouverte, sinon toast + son + point « non lu »
Conversation créée	`{"type": "conversation", "data": {…}}`	Apparaît dans la liste
Participant ajouté, groupe renommé	`{"type": "conversation", "data": {…}}`	La conversation est remplacée par sa nouvelle version

L'étiquette `type` permet au frontend de savoir ce qu'il reçoit. L'expéditeur reçoit aussi son propre message (par la réponse HTTP **et** par le WebSocket) : le frontend ignore un élément dont l'identifiant est déjà dans la liste, ce qui évite les doublons et garde à jour ses autres onglets ouverts.

Cycle de vie d'une notification. Quand un message arrive pour une conversation qui n'est pas ouverte, le frontend affiche un toast (auteur + début du message, avec le nom du groupe si c'est un groupe), joue un son, et ajoute un point doré à la conversation dans la liste. Le point disparaît quand l'utilisateur ouvre la conversation ; un clic sur le toast l'ouvre directement.

4 Fonctionnalités implémentées

Les captures suivantes ont été prises avec des comptes de démonstration.

4.1 MVP obligatoire

Conversations 1:1 et de groupe

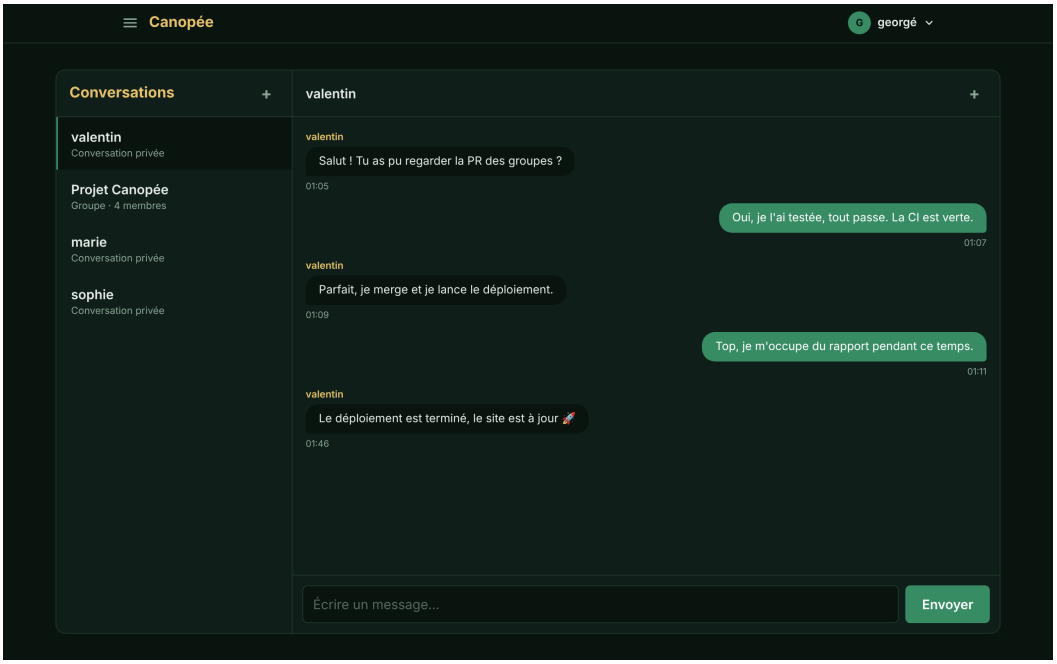


Fig. 3. – Conversation privée : mes messages à droite en vert, ceux de l'autre à gauche avec son pseudo.

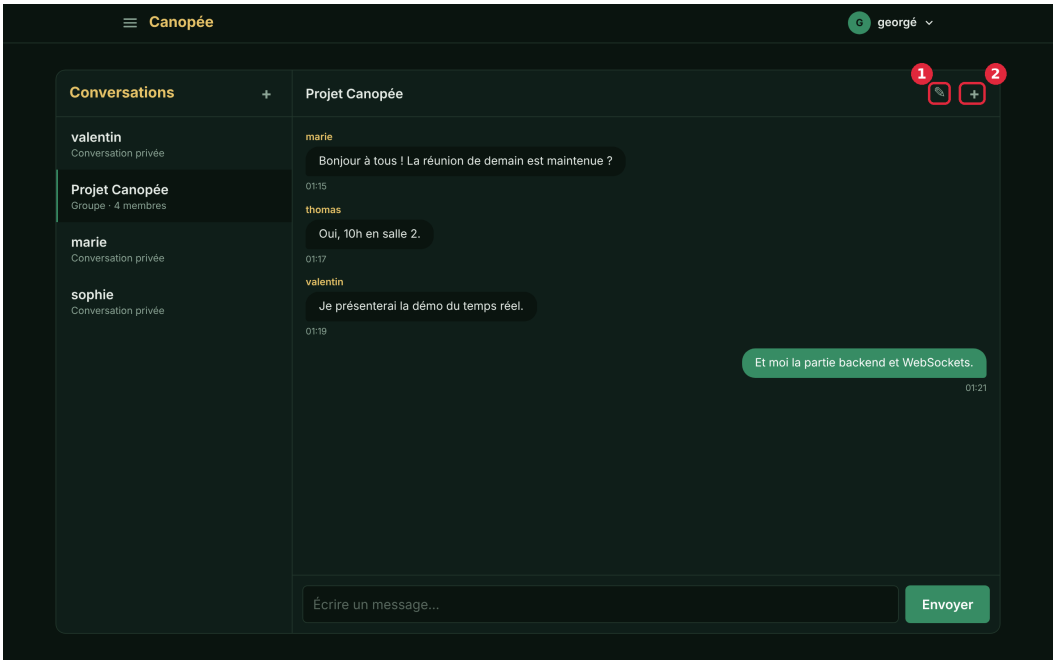


Fig. 4. – Conversation de groupe nommée « Projet Canopée » (4 membres) : (1) renommer le groupe, (2) ajouter un participant.

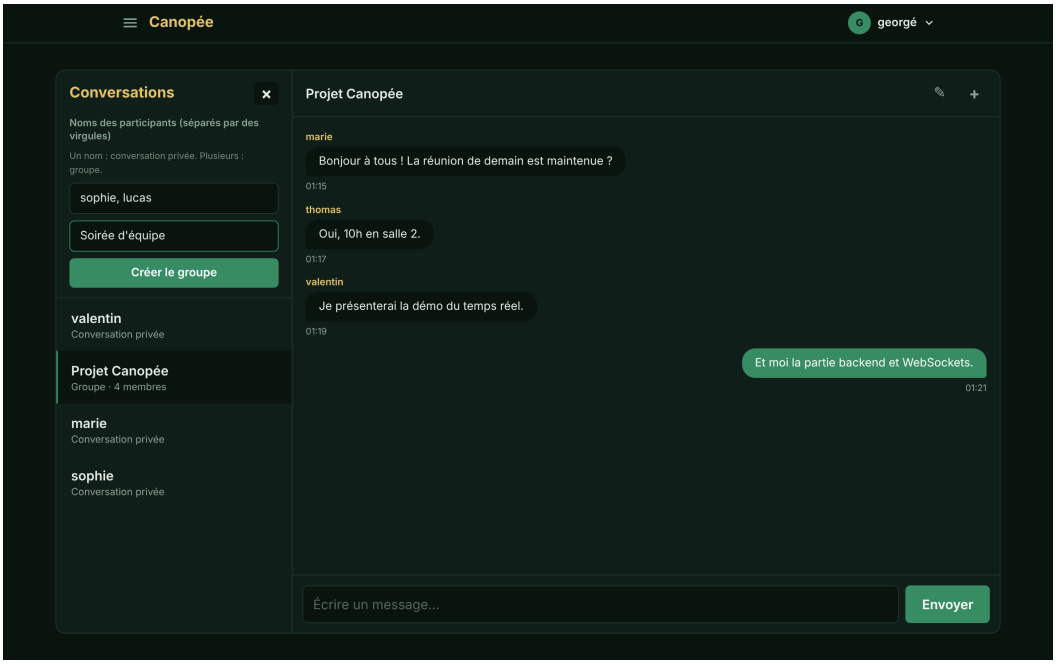


Fig. 5. – Création d'une conversation : un pseudo donne une conversation privée ; dès qu'il y en a plusieurs, le champ « Nom du groupe » apparaît.

Messages en temps réel

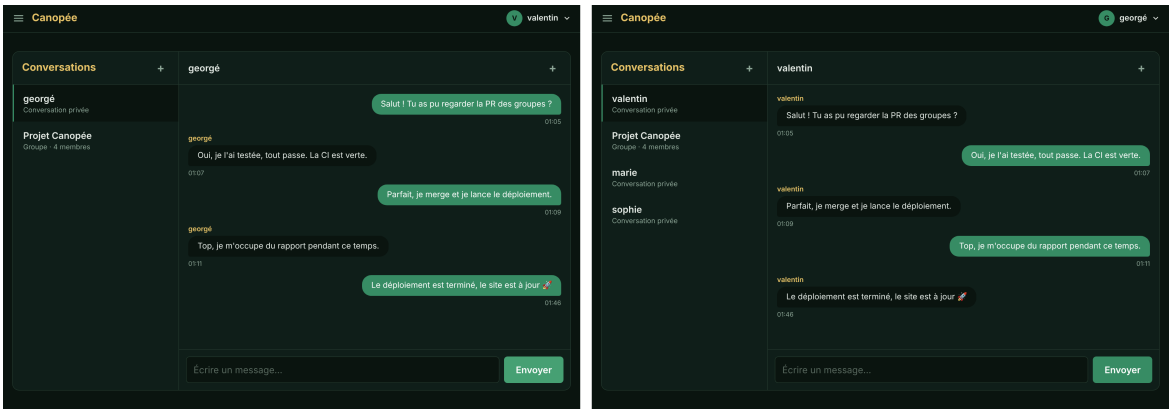


Fig. 6. – Deux navigateurs côte à côte : Valentin (à gauche) envoie un message, il apparaît immédiatement chez Georgé (à droite), sans rechargement.

Notifications temps réel

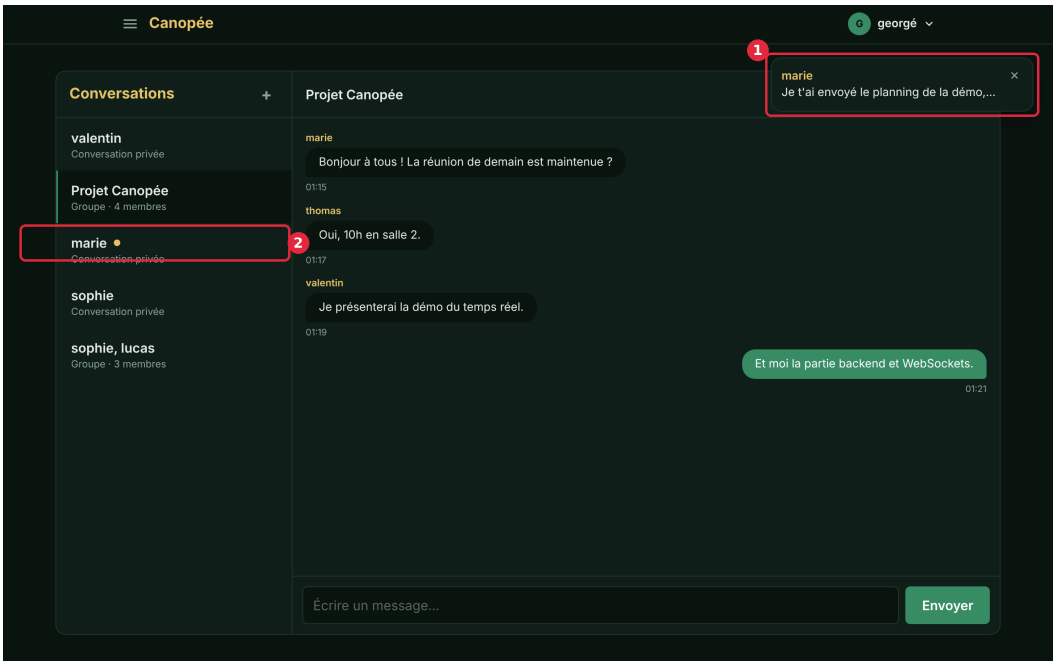


Fig. 7. – Georgé est dans « Projet Canopée » quand Marie lui écrit en privé : (1) toast (avec un son), (2) point « non lu » sur la conversation.

Authentification & permissions

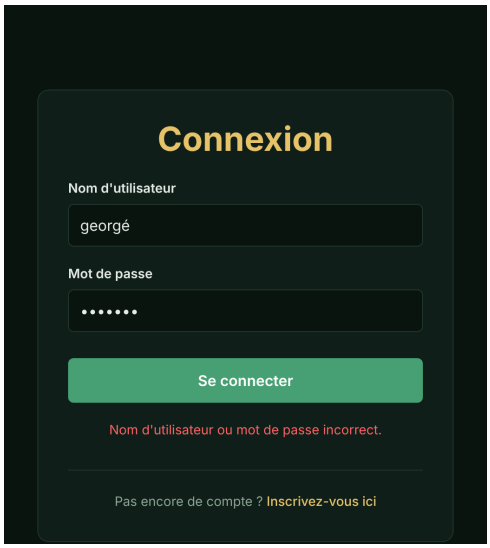


Fig. 8. – Mauvais identifiants : message clair.

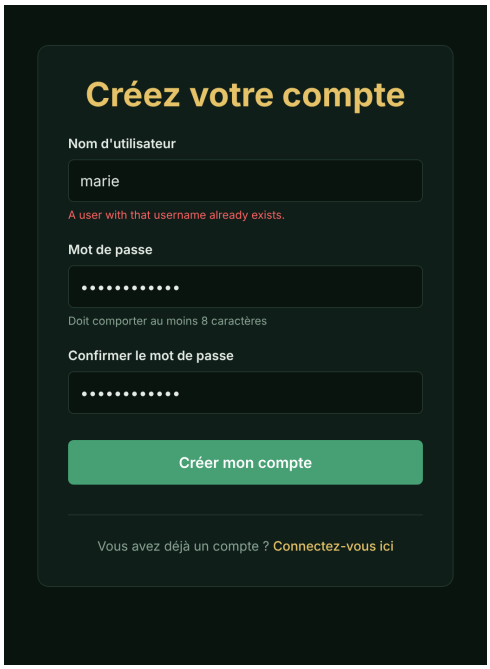


Fig. 9. – Inscription avec un pseudo déjà pris.

Un visiteur qui ouvre `/chat` sans être connecté est redirigé vers la page de connexion. Côté API, une requête sans token reçoit une erreur 401, et une conversation dont on ne fait pas partie renvoie une erreur 404.

Interface responsive

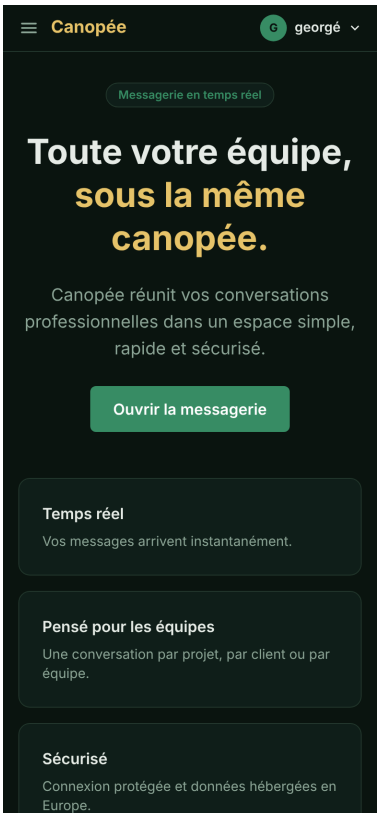


Fig. 10. – Accueil sur mobile.

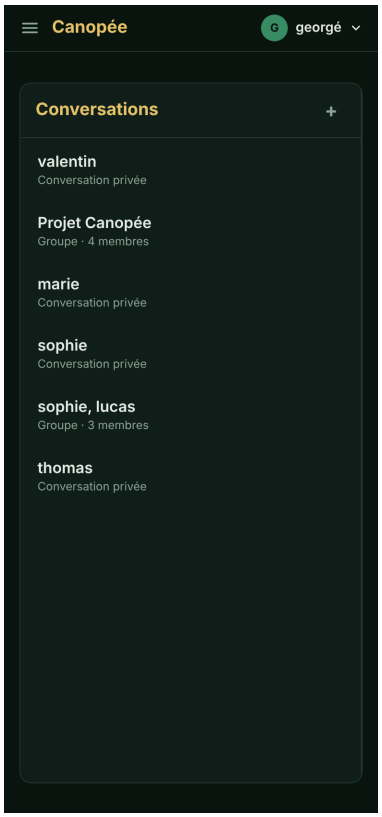


Fig. 11. – Liste des conversations.



Fig. 12. – Conversation ouverte.

Sur ordinateur, la liste et la conversation sont côte à côte. Sur un petit écran, une seule colonne est affichée à la fois, avec un bouton « ← » pour revenir à la liste.

4.2 Fonctionnalités optionnelles

Gestion des groupes

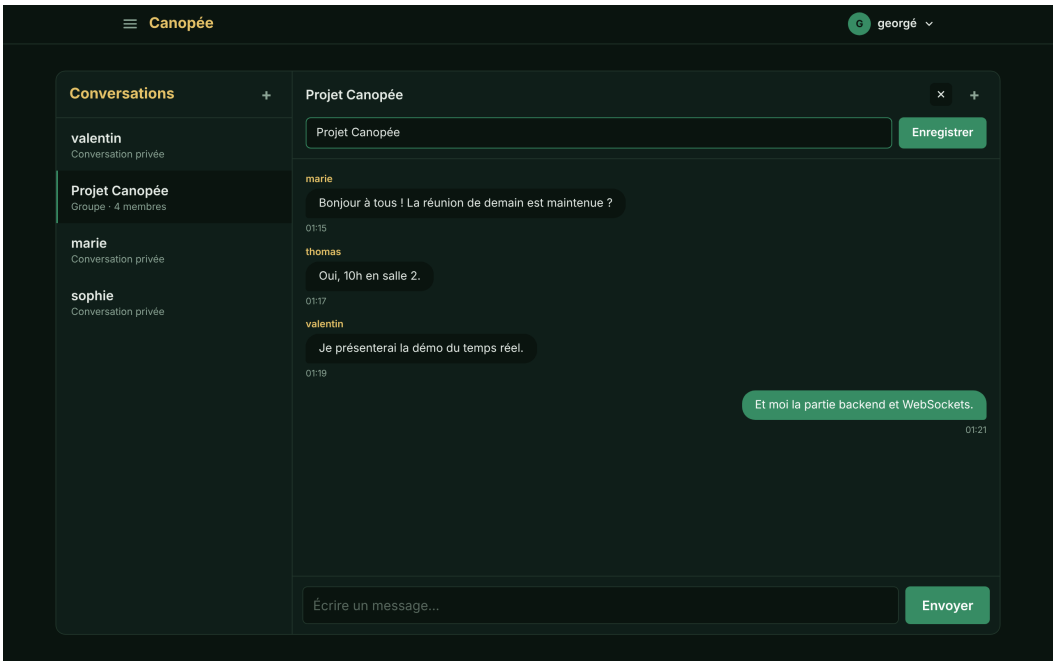


Fig. 13. – Renommage d'un groupe (✎). Le nouveau nom apparaît en direct chez tous les membres ; un nom vide réaffiche la liste des membres.

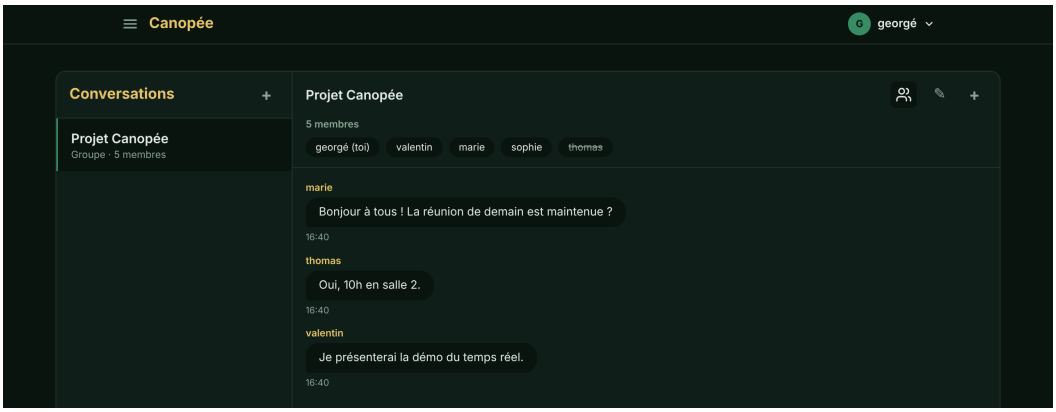


Fig. 14. – Le bouton « membres » affiche les participants du groupe ; un compte supprimé apparaît barré.

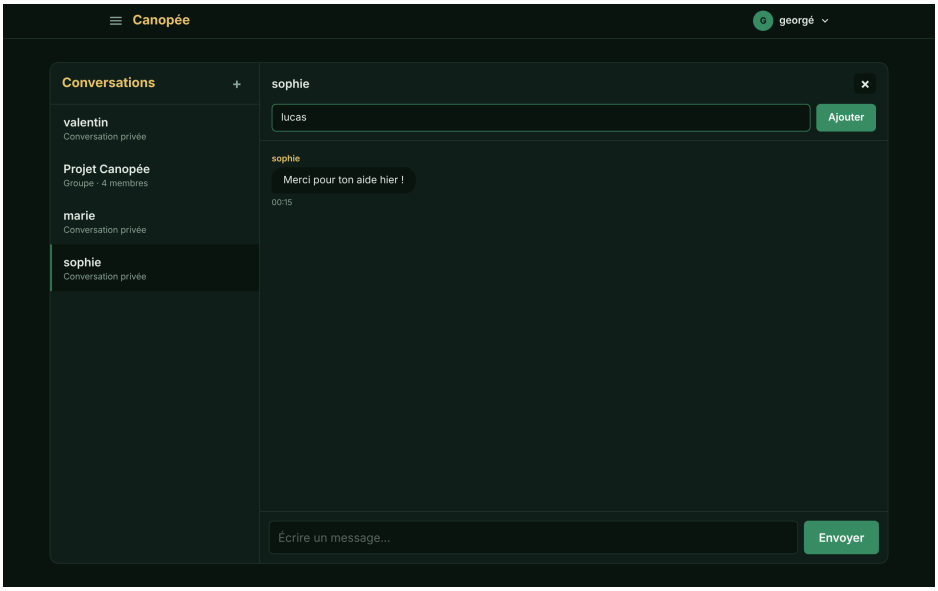


Fig. 15. – Dans la conversation privée avec Sophie, le « + » propose de créer un groupe : ajout de Lucas…

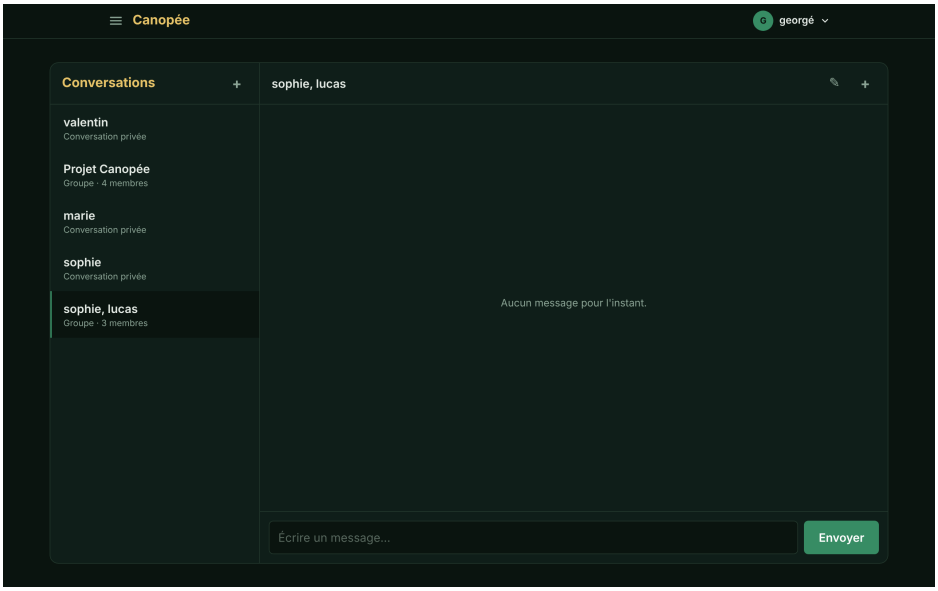


Fig. 16. – …ce qui crée et ouvre un nouveau groupe « sophie, lucas » ; la conversation privée avec Sophie ne change pas.

Impact sur la base de données. Deux champs ont été ajoutés à `Conversation` (`is_group`, `name`) par la migration `0002`, et la migration de données `0003` a corrigé les conversations existantes. L'ajout dans une conversation privée crée une nouvelle ligne dans `chat_conversation` et trois lignes dans la table de liaison ; la conversation privée et ses messages ne changent pas. Le renommage ne met à jour qu'une colonne (`save(update_fields=["name"])`).

Suppression de compte

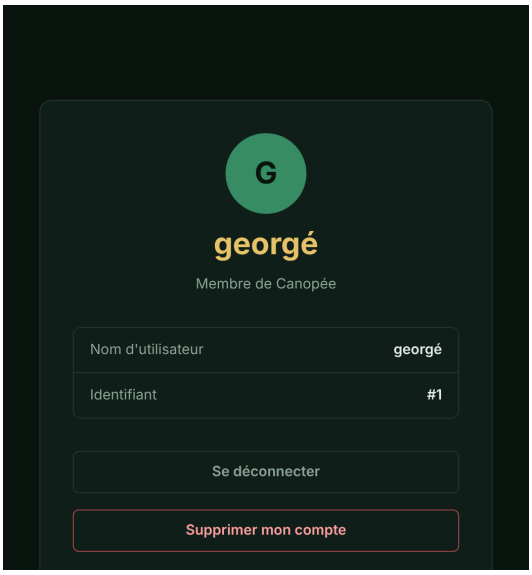


Fig. 17. – Page profil avec « Supprimer mon compte » (le mot de passe est demandé pour confirmer).

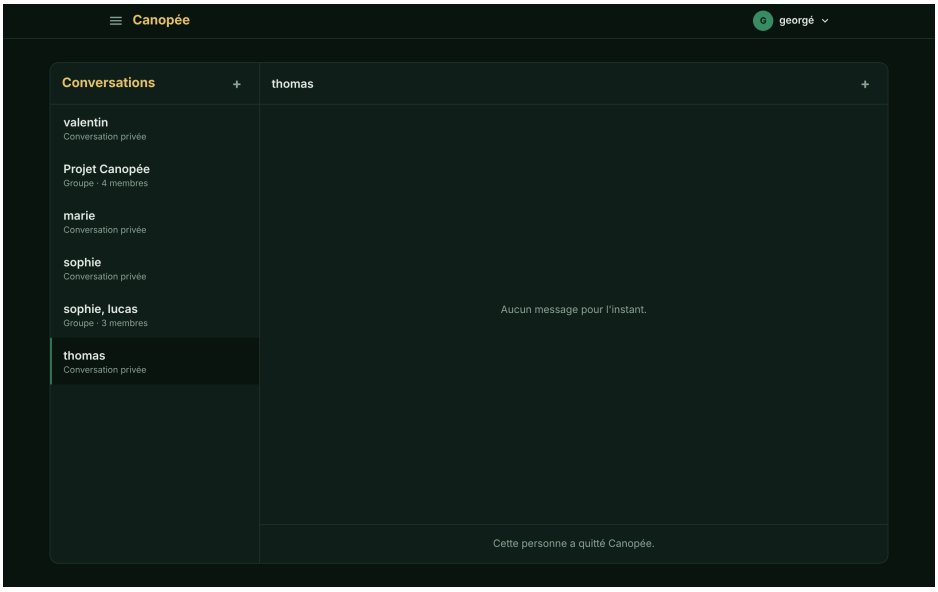


Fig. 18. – Thomas a supprimé son compte : la zone d'envoi est remplacée par « Cette personne a quitté Canopée ».

Impact sur la base de données. Aucune ligne n'est supprimée : `is_active` passe à faux et le mot de passe devient inutilisable. Les messages restent donc visibles par les autres participants. Le backend refuse ensuite de créer une conversation, d'ajouter ou d'écrire à ce compte.

4.3 Landing page & pricing

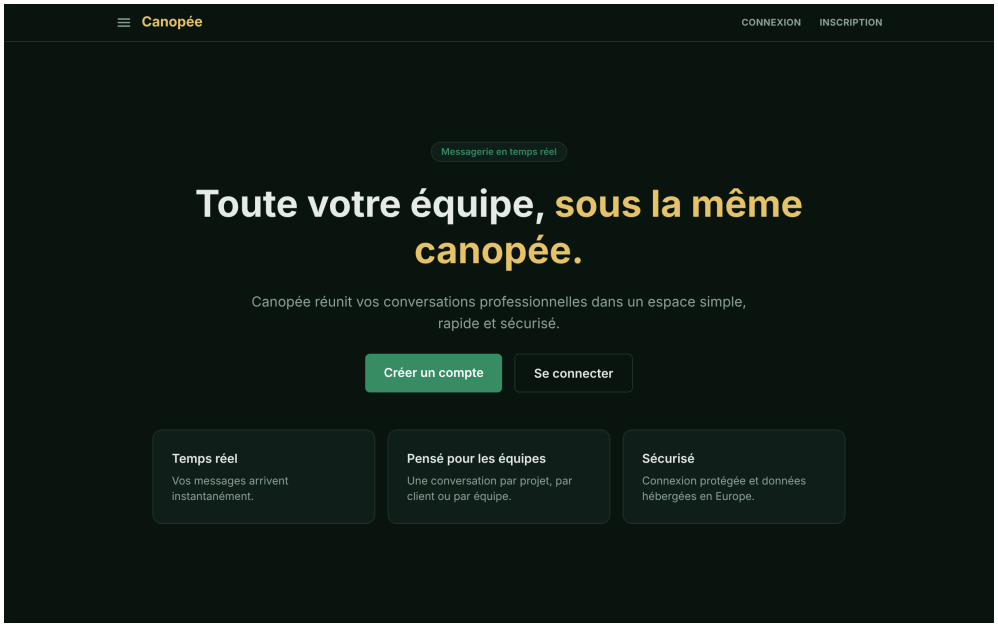


Fig. 19. – Landing page : promesse du produit et appels à l'action « Créer un compte » / « Se connecter ».

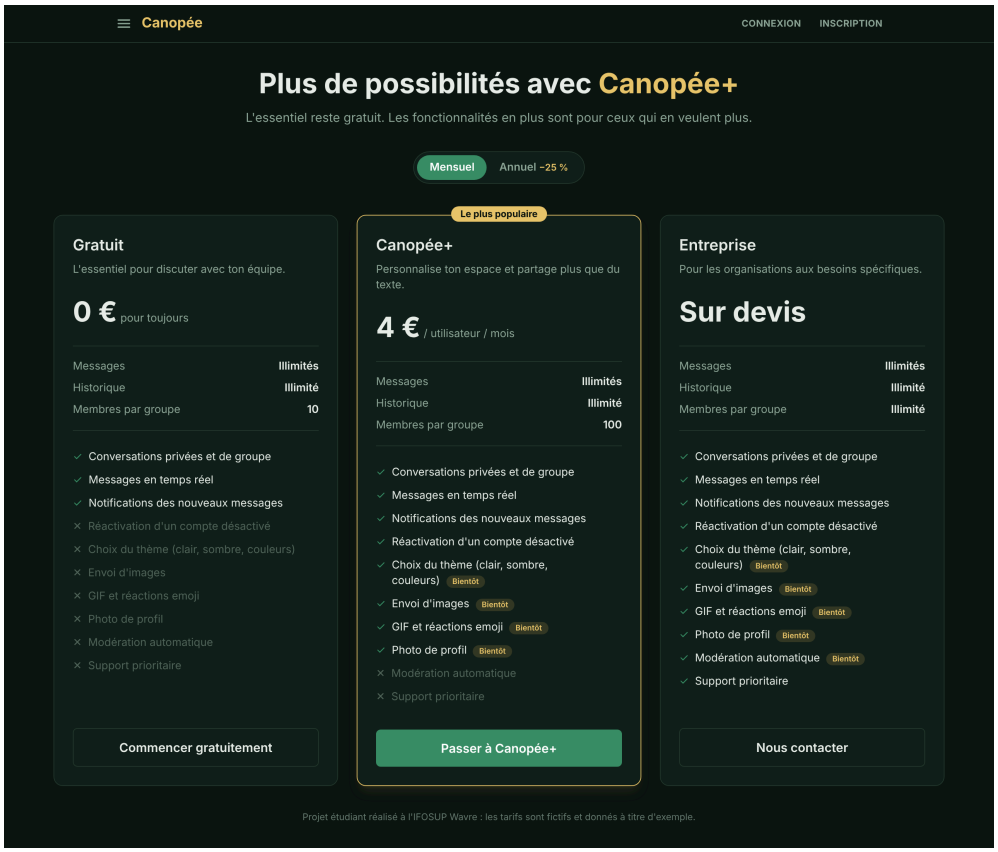


Fig. 20. – Grille tarifaire avec bascule mensuel / annuel.

Palier	Prix	Membres par groupe	Messages / historique
Gratuit	0 €	10	Illimités
Canopée+	4 € / utilisateur / mois (3 € en annuel)	100	Illimités
Entreprise	Sur devis	Illimité	Illimités

Justification des quotas. Un message texte ne prend presque pas de place en base : limiter le nombre de messages ou l'historique serait frustrant pour l'utilisateur pour un gain de stockage négligeable. La limite porte donc sur la **taille des groupes**, qui reflète la taille de l'équipe cliente : 10 membres suffisent à une petite équipe pour essayer gratuitement, une entreprise plus grande passe à Canopée+. Les paliers payants se distinguent aussi par des fonctionnalités (thèmes, images, GIF, photo de profil, modération) marquées « bientôt » sur la page : le paiement n'est pas encore implémenté.

5 CI/CD & déploiement

5.1 Workflow Git : ce que nous avons appris

Ce projet était notre premier vrai travail d'équipe avec Git, et c'est sans doute ce que nous avons le plus appris. Nous avons suivi un **Git Flow simplifié** :

- `main` : uniquement du code stable, déployé en production ; protégée, impossible d'y pousser directement ;
- `develop` : la branche d'intégration, où arrivent les fonctionnalités terminées ;
- `feature/<nom>` : une branche par fonctionnalité, créée depuis un `develop` à jour, puis fusionnée par une **Pull Request** avec une description et des étapes de test.

Les messages de commit suivent la convention `type: description` (`feat`, `fix`, `docs`, `chore`), ce qui rend l'historique lisible d'un coup d'œil. Au 2 novembre : **95 commits** et **32 Pull Requests** fusionnées dans `develop`, sans compter celle de ce rapport.

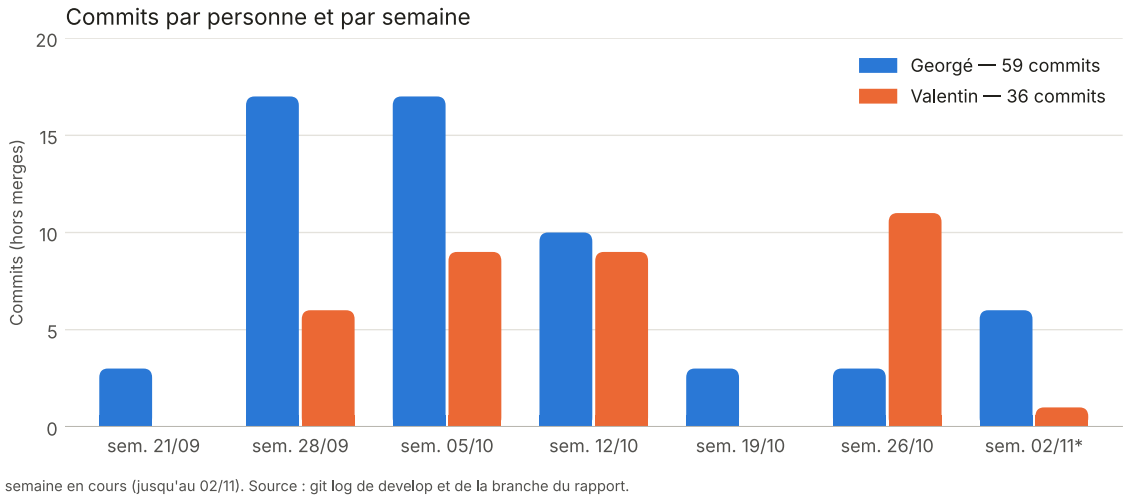


Fig. 21. – Commits par personne et par semaine (hors commits de merge).

Membre	feat	fix	docs	chore / autre	Total
van der Veen Georgé	31	8	15	5	59
Coisne Valentin	29	4	1	2	36

Le graphique montre notre organisation : Georgé a posé les fondations (modèles, API, WebSockets) pendant les premières semaines, puis Valentin a pris le relais sur l'interface, le déploiement et le déploiement continu fin octobre. Les nombreux commits `docs` de Georgé correspondent au devlog (`docs/DEVLOG.md`), tenu à jour à chaque fonctionnalité.

Ce que nous avons appris, souvent en faisant des erreurs : toujours partir d'un `develop` à jour (`git pull`) avant de créer une branche ; résoudre un conflit de merge sans réintroduire de code en double (difficulté n°7) ; vérifier qu'un commit est bien **poussé** (`git status` affiche `[ahead 1]` sinon) ; mettre ses changements de côté avec `git stash` ; fusionner avec « Create a merge commit » plutôt qu'en « squash » pour que `develop` et `main` gardent le même historique ; et désactiver la suppression automatique des branches sur GitHub, qui avait supprimé `develop` après une fusion dans `main`.

5.2 Intégration continue (à chaque Pull Request)

À chaque Pull Request vers `develop` ou `main`, GitHub Actions lance deux jobs en parallèle sur une machine neuve :

- **Backend** : installation des dépendances avec `uv sync`, puis `manage.py test`.
- **Frontend** : `npm ci`, le linter (`npm run lint`), puis le build de production (`npm run build`).

Une Pull Request qui casse un test, le lint ou le build affiche une croix rouge avant même d'être relue.

5.3 Déploiement continu (au merge sur `main`)

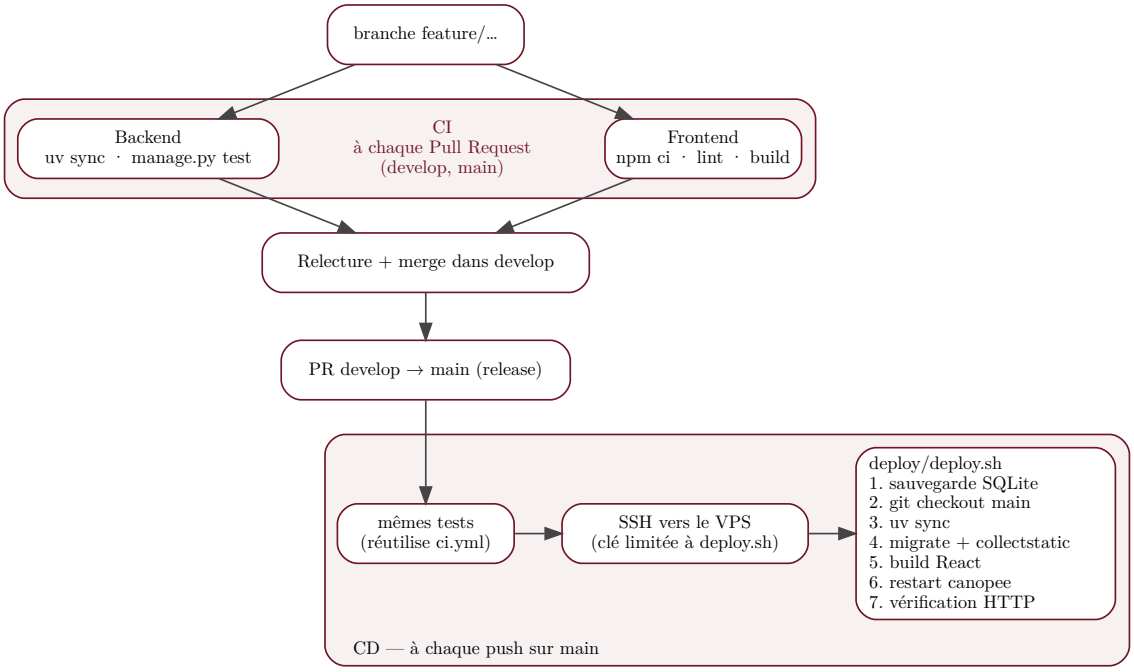


Fig. 22. – Pipeline CI/CD : de la branche de fonctionnalité à la production.

Chaque push sur `main` (donc chaque Pull Request fusionnée) relance les mêmes tests, puis, s'ils passent, se connecte en SSH au VPS. La clé SSH utilisée est limitée à une seule action : lancer `deploy/deploy.sh`. Ce script, écrit pour s'arrêter à la première erreur :

1. sauvegarde la base SQLite (les 10 dernières sauvegardes sont gardées) ;
2. récupère la dernière version de `main` ;
3. installe les dépendances Python (`uv sync --frozen`) ;
4. applique les migrations et collecte les fichiers statiques ;
5. compile le frontend React et le copie dans le dossier servi par Nginx ;
6. redémarre le service `canopee` ;
7. vérifie que le backend **et** le site public répondent ; sinon, il affiche les derniers logs et échoue.

Deux déploiements ne peuvent pas tourner en même temps (un verrou côté GitHub et côté serveur).

Secrets. Aucun secret n'est dans le dépôt. Côté serveur, Django lit sa configuration dans un fichier `.env` ignoré par Git (`DJANGO_SECRET_KEY`, `DJANGO_DEBUG`, hôtes et origines autorisés) ; un fichier `.env.example` documente les variables attendues. Django refuse de démarrer en production si la clé secrète n'a pas été définie. Côté GitHub, la clé SSH et l'adresse du serveur sont des **secrets** du dépôt.

Logs. Le déroulé de chaque déploiement est visible dans l'onglet **Actions** de GitHub ; les logs du backend sont accessibles sur le serveur avec `journalctl -u canopee`.

Choix de l'hébergeur. Le VPS répond aux trois exigences : **persistance** (la base SQLite est sur son disque et survit aux redémarrages et aux déploiements), **WebSockets** (Nginx relaie les connexions `/ws/` avec un délai d'inactivité d'une heure, sans mise en veille

du service) et **maîtrise des coûts** (un petit VPS suffit pour notre charge, sans dépendre des limites d'un free tier).

6 Qualité, tests & gestion des erreurs

Lint et formatage. Le frontend est vérifié par `oxlint` dans la CI. Côté Python, le code est formaté automatiquement par l'éditeur ; nous n'avons pas encore ajouté de linter Python à la CI.

Documentation. Le `README` présente le projet et explique comment le lancer en local en quelques commandes (backend avec `uv`, frontend avec `npm`) ; le dossier `docs/` contient le cahier des charges, le diagramme de classes, le devlog et la todolist. Le fichier `backend/.env.example` liste les variables nécessaires en production.

Tests. Les 10 tests backend portent sur les points les plus sensibles : les **droits d'accès** et la **suppression de compte**.

```
test_participant_sees_their_conversation ..... ok
test_non_participant_does_not_see_conversation ..... ok
test_unauthenticated_user_cannot_list_conversations .... ok
test_participant_can_read_messages ..... ok
test_non_participant_cannot_read_messages ..... ok
test_unauthenticated_user_cannot_read_messages ..... ok
test_wrong_password_is_refused ..... ok
test_account_is_deactivated_and_keeps_its_name ..... ok
test_deleted_user_cannot_log_in ..... ok
test_cannot_write_to_a_deleted_user ..... ok
Ran 10 tests — OK
```

Chaque test vérifie une seule règle, dans une base de test vide créée par Django : un participant voit sa conversation, un intrus ne voit rien, un visiteur non connecté reçoit une erreur 401, un compte supprimé ne peut plus se connecter, etc.

Ce qui n'est pas encore testé, et que nous testerions en priorité : l'envoi d'un message (`POST`) avec ses cas d'erreur, les règles des groupes (pas de doublon de conversation privée, l'ajout dans une privée crée un groupe, seul un groupe peut être renommé), et le consumer WebSocket (connexion refusée sans token, réception d'un message), avec l'outil de test fourni par Channels. Ces fonctionnalités ont été testées à la main, avec plusieurs navigateurs connectés à des comptes différents.

Gestion des erreurs et des cas limites.

- Toutes les erreurs de l'API sont des réponses JSON avec un message (400, 401, 404), jamais une page d'erreur technique ; en production, `DEBUG` est désactivé.
- Le frontend traduit ces erreurs en messages compréhensibles : « Nom d'utilisateur ou mot de passe incorrect », « Utilisateur introuvable », « Cette personne a quitté Canopée »…
- Un token expiré est renouvelé automatiquement ; si le renouvellement échoue aussi (après 3 jours), l'utilisateur est renvoyé vers la page de connexion.
- Un double clic sur « Envoyer » n'envoie pas le message deux fois (le bouton est désactivé pendant l'envoi).

- Un nom de groupe trop long est refusé avec un 400 et un message, au lieu de faire planter la base.

7 Difficultés rencontrées & solutions

Nous avons tenu un journal de développement (`docs/DEVLOG.md`) tout au long du projet, avec le code, les explications et les erreurs rencontrées. Voici les problèmes les plus marquants.

1. Le frontend n'affichait que des numéros à la place des pseudos
Symptôme. la liste des conversations affichait `[1, 2]` et les messages `author: 2` : impossible de savoir qui était qui.
Cause. par défaut, Django REST Framework représente une relation vers un autre objet par son identifiant.
Solution. un serializer imbriqué (`UserSerializer`) renvoie chaque participant sous la forme `{id, username}`, et un champ `author_name` donne le pseudo de l'auteur. Comme un serializer imbriqué est en lecture seule, la création utilise un second champ, `participant_usernames`.

2. Déconnexion au bout de 5 minutes
Symptôme. après quelques minutes, plus aucune action ne fonctionnait et l'application demandait de se reconnecter.
Cause. sans configuration, SimpleJWT donne au token d'accès une durée de vie de 5 minutes, et le frontend n'utilisait pas le token de refresh.
Solution. durée du token d'accès portée à 30 minutes, et le frontend renouvelle automatiquement le token quand une requête reçoit une erreur 401, puis la rejoue. Le WebSocket se reconnecte aussi avec un token neuf.

3. Les nouvelles conversations n'apparaissaient pas chez l'autre
Symptôme. quand Alice créait une conversation avec Bob, Bob devait recharger la page pour la voir.
Cause. seuls les messages passaient par le WebSocket ; la liste des conversations n'était chargée qu'une fois, à l'ouverture de la page.
Solution. le serveur envoie aussi les conversations créées ou modifiées. Chaque événement porte désormais une étiquette (`message` ou `conversation`), ce qui a demandé de changer le backend et le frontend en même temps.

4. Les messages reçus en direct ne s'affichaient pas (piège des closures React)

Symptôme. les messages arrivaient bien par le WebSocket, mais n'apparaissaient pas, ou remplaçaient toute la liste.
Cause. la fonction qui reçoit les événements est créée une seule fois, au chargement de la page : elle « voyait » pour toujours les valeurs de départ (aucune conversation ouverte, liste vide).
Solution. lire la conversation ouverte dans un `useRef`, et mettre à jour les listes avec la forme fonctionnelle `setMessages((prev) => …)`, qui reçoit toujours la valeur à jour.

5. Une route qui « n'existe pas » alors qu'elle est dans le code

Symptôme. la page d'erreur de Django listait les routes connues, sans la nouvelle route d'ajout de participant, pourtant bien présente dans `urls.py`.
Cause. un ancien serveur de développement, lancé avant de changer de branche, tournait encore dans un autre terminal et répondait à sa place.
Solution. arrêter tous les serveurs, puis relancer depuis le bon dossier. Leçon : un serveur garde le code chargé au démarrage.

6. Recréer une conversation privée ouvrait le mauvais groupe

Symptôme. créer une conversation avec quelqu'un qui en avait déjà une avec nous ouvrait un ancien groupe au lieu de la conversation privée.
Cause. la migration qui a ajouté `is_group` a donné `False` à **toutes** les conversations existantes, y compris les anciens groupes. Le backend trouvait donc d'abord un ancien groupe « marqué privé ».
Solution. une migration de données (`0003`) qui marque comme groupes les conversations de plus de deux participants. Elle s'applique aussi automatiquement sur le serveur au déploiement. Leçon : quand on ajoute un champ, il faut penser aux données déjà en base.

7. Le frontend ne compilait plus après un merge

Symptôme. `npm run dev` affichait `PARSE_ERROR: Unexpected token` à la dernière ligne de `chat.js`.
Cause. en résolvant un conflit de merge entre deux branches, la fin d'une fonction avait été gardée deux fois, laissant une accolade en trop.
Solution. suppression de l'accolade. Depuis, nous relançons le build après chaque résolution de conflit, et la CI le vérifie de toute façon avant chaque fusion.

8. Un commit « disparu » et une branche `develop` supprimée
Symptôme. une correction faite en local n'apparaissait sur aucune branche GitHub ; plus tard, la branche `develop` avait disparu de GitHub.
Cause. le commit n'avait jamais été poussé ; et l'option « Automatically delete head branches » de GitHub supprime la branche source après la fusion d'une Pull Request, y compris `develop` après une fusion dans `main`.
Solution. `git push`, et `git status -sb` pour vérifier qu'il ne reste rien à pousser ; recréation de `develop` depuis `main` et désactivation de l'option.

8 Utilisation de l'IA

Nous avons utilisé l'assistant **Claude** (Anthropic) tout au long du projet, pour des tâches différentes selon les membres. Le détail individuel figure dans le carnet métacognitif de chacun.

- **Apprentissage guidé.** Pour le backend, Georgé a utilisé Claude comme un tuteur : il lui demandait d'avancer une étape à la fois, avec la structure du code et des indices, mais sans la solution, puis lui soumettait son propre code pour qu'il le corrige. La plupart du code backend (modèles, API, WebSockets, groupes) a été écrit de cette façon.
- **Génération de code.** Sur demande explicite, Claude a écrit certaines parties du frontend de la page de chat (réception en temps réel, ajout de participants, groupes, renommage) ; les commits concernés le mentionnent (« written by Claude »).
- **Débogage.** Analyse de messages d'erreur et de comportements inattendus (difficultés n°5 à 7).
- **Documentation.** Aide à la rédaction et à la relecture du devlog, des descriptions de Pull Requests et des rapports ; les captures d'écran et les schémas de ce rapport ont été générés avec son aide, à partir de l'application réelle et de l'historique Git.

Comment nous avons validé les sorties. Aucun code n'a été intégré sans passer par une Pull Request : la CI relance les tests, le lint et le build, et l'autre membre relit les changements. Chaque fonctionnalité a été testée à la main, avec au moins deux navigateurs connectés à des comptes différents pour le temps réel. Pour les parties apprises avec l'IA, le devlog réexplique chaque morceau de code avec nos mots, ce qui nous oblige à le comprendre et nous permet de le défendre ; en cas de doute, nous nous sommes appuyés sur la documentation officielle de Django, Django REST Framework et Channels.

9 Réflexion, améliorations & conclusion

Ce que nous avons appris. Techniquement, ce projet nous a appris à construire une API REST sécurisée avec Django, à faire du vrai temps réel avec les WebSockets, et à faire communiquer deux applications séparées. Mais le plus gros apprentissage est ailleurs : travailler à deux sur le même code avec Git, des Pull Requests et une CI, et mettre une application en production avec un déploiement automatique. Nous avons aussi compris l'intérêt de tenir un journal : il a rendu ce rapport bien plus simple à écrire.

Avec plus de temps, nous ajouterions :

- un vrai **compteur de non-lus**, conservé après rechargement : un champ `last_read_at` par participant, ce qui demande de transformer la relation N-N en modèle explicite (`through=`) ;
- un **administrateur de groupe**, la possibilité de **quitter un groupe**, et le tri des conversations par dernier message ;
- des **tests** pour l'envoi de messages, les règles des groupes et le WebSocket ;
- l'indicateur « en train d'écrire… » et le statut en ligne, qui réutiliseraient la connexion WebSocket existante ;
- Redis et PostgreSQL seulement si le nombre d'utilisateurs l'exigeait.

Ce que nous ferons différemment au Projet 2 : écrire les tests en même temps que chaque route, et non à la fin ; réfléchir au modèle de données complet dès le début (le type de conversation dès la première migration nous aurait évité la difficulté n°6) ; et faire des Pull Requests plus petites et plus fréquentes, pour limiter les conflits de merge.

9.1 Glossaire

Terme	Définition
API REST	Ensemble d'URL que le frontend appelle en HTTP (`GET`, `POST`, `PATCH`…) pour lire ou modifier des données, échangées en JSON.
ASGI	Asynchronous Server Gateway Interface : la façon dont un serveur comme Daphne fait tourner Django en gérant aussi les connexions longues comme les WebSockets (contrairement à WSGI, limité à « une requête, une réponse »).
Channel layer	Système de messagerie interne de Django Channels, qui permet d'envoyer un événement à un groupe de connexions WebSocket.
CI/CD	Intégration continue (tester automatiquement chaque modification) et déploiement continu (mettre en production automatiquement ce qui est fusionné dans `main`).
Consumer	L'équivalent d'une vue Django pour une connexion WebSocket : une classe dont les méthodes sont appelées à la connexion, à la réception d'un événement et à la déconnexion.
JWT	JSON Web Token : jeton signé qui prouve l'identité de l'utilisateur, envoyé avec chaque requête dans l'en-tête `Authorization`.
Migration	Fichier versionné qui décrit une modification du schéma (ou des données) de la base, appliqué avec `manage.py migrate`.
ORM	Object-Relational Mapper : permet de manipuler la base de données avec des objets Python au lieu d'écrire du SQL.
Pull Request	Demande de fusion d'une branche dans une autre sur GitHub, avec description, relecture et vérifications de la CI.
Serializer	Classe de Django REST Framework qui valide les données reçues et convertit les objets en JSON.
WebSocket	Connexion qui reste ouverte entre le navigateur et le serveur, dans les deux sens : le serveur peut envoyer des données sans qu'on les lui demande.